ESPECIFICAÇÃO B1

Registro de Programa de Computador

PlantUML Parser HSL com Saída Estruturada para Casos de Uso

Produto 4 - Versão candidata 1.0.0

Autores	Helaine Solange Lins Barreiros Ivaldir Honório de Farias Júnior Cleyton Mário de Oliveira Rodrigues

Obra originária	`Enteee/plantuml-parser`
Licença	Apache License 2.0
Commit-base	`966a72e`
Commit funcional	`f317b79`
Branch	`codex/registro-parser-usecase-v1.0.0`
Estado	Minuta técnica completa; pacote e SHA-256 ainda não gerados

30 de outubro de 2026

O objeto é a versão derivada que acrescenta suporte estruturado a diagramas de casos de uso. O pedido não reivindica autoria ou exclusividade sobre o código preexistente. A licença, a obra originária e o escopo das modificações devem permanecer identificados.

Sumário

1 Introdução **1**

1.1 Finalidade . . . 1

1.2 Contexto de desenvolvimento . . . 1

1.3 Natureza derivada . . . 1

2 Identificação **1**

3 Problema, finalidade e usuários **1**

3.1 Problema atendido . . . 1

3.2 Finalidade . . . 2

3.3 Usuários-alvo . . . 2

4 Fronteira do produto **2**

4.1 Escopo incluído nesta versão . . . 2

4.2 Escopo explicitamente excluído . . . 2

4.3 Arquivos modificados ou acrescentados desde o commit-base . . . 2

5 Funcionalidades principais **3**

6 Arquitetura e contratos **3**

6.1 Entradas . . . 3

6.2 Saídas . . . 4

6.3 Relação com o Produto 3 . . . 4

7 Proveniência e autoria **4**

7.1 Obra originária . . . 4

7.2 Modificações desta versão . . . 4

7.3 Contribuições posteriores fora do objeto . . . 4

8 Originalidade e alcance dos direitos **5**

9 Classificação candidata **5**

10 Privacidade e configuração externa **5**

11 Pacote depositável e forma de entrega **5**

11.1 Conteúdo a incluir . . . 5

11.2 Conteúdo a excluir . . . 6

11.3 Artefatos de entrega 6

12 Verificação e limitações **6**

13 Resumo digital **7**

14 Pendências institucionais **7**

1 Introdução

1.1 Finalidade

O programa analisa texto PlantUML e fornece uma representação estruturada, em JSON, de diagramas de casos de uso. A saída separa atores, casos de uso, pacotes, retângulos e relações, preservando atributos relevantes para processamento automatizado por outros programas.

1.2 Contexto de desenvolvimento

A extensão foi desenvolvida para apoiar a avaliação programática de diagramas UML no doutorado. O parser originário já realizava parsing de PlantUML; a versão HSL modifica e amplia essa base para reconhecer construções necessárias aos casos de uso e emitir uma saída organizada para consumo pelo motor de avaliação.

1.3 Natureza derivada

A versão conserva arquitetura, gramática e código do projeto originário Enteee/plantuml-parser, licenciado sob Apache-2.0, e modifica essa base. Por isso, tecnicamente é obra derivada. Essa qualificação descreve a proveniência e não impede que as modificações autorais formem um produto relacionado e um pedido próprio, desde que a origem e a licença sejam respeitadas.

2 Identificação

Campo	Informação
Nome do programa	PlantUML Parser HSL com Saída Estruturada para Casos de Uso
Nome curto	PlantUML Parser HSL - Use Case Structured
Versão	v1.0.0 candidata; confirmar após o congelamento
Data de criação	Pendente de validação; marco funcional no commit `f317b79`
Data de publicação	Pendente de orientação do NIT; não inferir automaticamente do GitHub
Linguagens	TypeScript, JavaScript e PEG.js
Ambiente	Biblioteca Node.js e executável de linha de comando
Natureza	Versão derivada de software livre, com modificações autorais identificadas
Licença	Apache-2.0, preservada da obra originária

3 Problema, finalidade e usuários

3.1 Problema atendido

A árvore sintática genérica de um parser não fornece necessariamente um contrato direto para avaliação de diagramas de casos de uso. O software acrescenta reconhecimento e formatação capazes de produzir categorias explícitas e relacionamentos estruturados, reduzindo o acoplamento dos consumidores à estrutura interna da árvore de parsing.

3.2 Finalidade

- reconhecer elementos de diagramas de casos de uso em PlantUML;
- representar atores e seus estereótipos;
- representar casos de uso, pacotes e retângulos;
- preservar a associação dos elementos aos grupos;
- estruturar relações, extremidades, corpos de seta, rótulos e tipo de relação;
- oferecer formatadores JSON consumíveis por ferramentas externas;
- disponibilizar a variante pelo módulo e pelo executável `plantuml-parser-hsl`.

3.3 Usuários-alvo

Desenvolvedores, pesquisadores e ferramentas que precisam transformar diagramas de casos de uso PlantUML em dados estruturados para validação, comparação, métricas ou integração.

4 Fronteira do produto

4.1 Escopo incluído nesta versão

- alterações de gramática para construções utilizadas em casos de uso;
- ampliação dos tipos internos para atores, casos de uso, grupos e relações;
- formatadores `structured` e `usecase_structured`;
- integração dos formatadores à API exportada;
- ajustes do formatador de grafo relacionados aos novos tipos;
- configuração de compilação, manifesto NPM e lockfile compatíveis;
- exemplo de diagrama e script de verificação existentes no commit da extensão.

4.2 Escopo explicitamente excluído

- extensões posteriores para diagramas de classe, sequência ou atividade;
- alterações posteriores aos commits `f317b79` e `8446cf4`;
- Motor de Avaliação Técnica e Estatística em Python;
- resultados, diagramas e planilhas empíricas do doutorado;
- `node_modules`, arquivos compilados, cobertura, caches e tokens;
- qualquer reivindicação sobre o código originário não modificado.

4.3 Arquivos modificados ou acrescentados desde o commit-base

O diff `966a72e..f317b79` identifica o núcleo da versão:

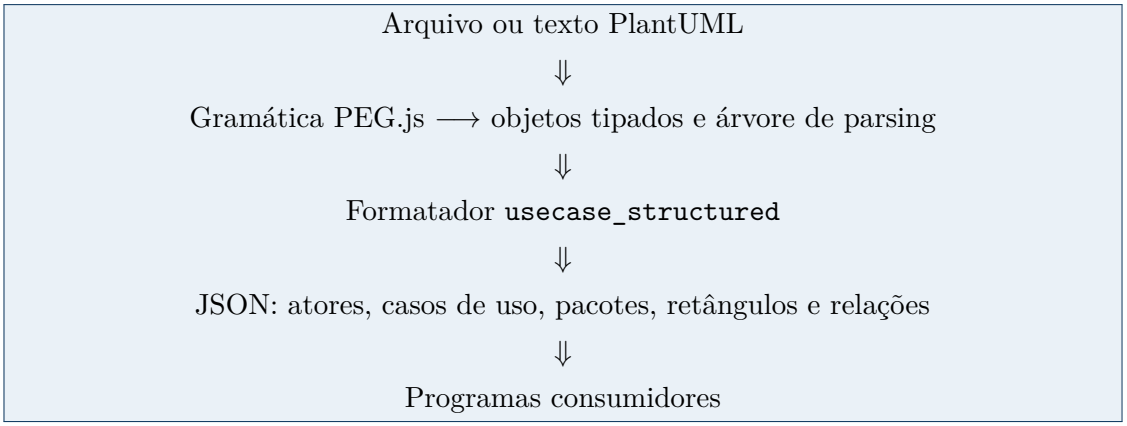
- `src/formatters/graph.ts`: ajustes de atores e relações;

- `src/formatters/structured.ts`: novo formatador estruturado;
- `src/formatters/usecase_structured.ts`: formatador especializado;
- `src/index.ts`: exportação e registro dos formatadores;
- `src/plantuml.pegjs`: extensões da gramática;
- `src/types.ts`: tipos para casos de uso, atores, grupos e relações;
- `package.json`, `package-lock.json` e `tsconfig.json`: manifesto, dependências e compilação;
- `actor_test.plantuml` e `test-actor.js`: entrada e verificação de apoio.

5 Funcionalidades principais

ID	Funcionalidade	Evidência principal
F1	Analisar fonte PlantUML usando a gramática estendida	`src/plantuml.pegjs`
F2	Representar atores, casos de uso e grupos por tipos próprios	`src/types.ts`
F3	Extrair atores e estereótipos	`usecase_structured.ts`
F4	Extrair casos de uso	`usecase_structured.ts`
F5	Organizar elementos contidos em pacotes e retângulos	`usecase_structured.ts`
F6	Estruturar relações e detalhes das setas	`usecase_structured.ts`
F7	Inferir tipos conhecidos das extremidades das relações	`usecase_structured.ts`
F8	Emitir JSON organizado por categorias	formatadores estruturados
F9	Expor os novos formatadores na biblioteca	`src/index.ts`
F10	Disponibilizar execução pelo binário NPM	`package.json`

6 Arquitetura e contratos



6.1 Entradas

- texto PlantUML ou caminho/glob de arquivos PlantUML;

- opções já suportadas pela API de parsing;
- seleção do formatador de saída.

6.2 Saídas

O JSON contém:

- `actors`: nome, título e estereótipos;
- `usecases`: nome e título;
- `packages` e `rectangles`: identificação e elementos internos;
- `relationships`: origem, destino, tipos conhecidos, tipo de relação, rótulo e partes das setas.

6.3 Relação com o Produto 3

O Produto 3 chama o executável `plantuml-parser-hsl` por meio de um adaptador Python e consome o JSON. Essa relação é de interoperabilidade: o código TypeScript do parser permanece neste repositório e não precisa ser copiado integralmente para o pacote do Produto 3.

7 Proveniência e autoria

7.1 Obra originária

- projeto: `Enteee/plantuml-parser`;
- repositório: `https://github.com/Enteee/plantuml-parser`;
- autor originário indicado no manifesto: Ente;
- licença: Apache License 2.0;
- commit-base adotado: `966a72e`.

7.2 Modificações desta versão

O histórico entre `966a72e` e `f317b79` contém um commit funcional atribuído a Helaine Barreiros. A documentação de saneamento foi acrescentada no commit `8446cf4`. O pedido deve identificar Helaine como autora das modificações desta versão, sem substituir os créditos ou direitos sobre a obra originária.

7.3 Contribuições posteriores fora do objeto

As extensões de classes, sequências e atividades existentes em outras branches ou commits posteriores não pertencem a esta versão. Seus autores não devem ser incluídos automaticamente neste pedido, salvo decisão de ampliar a fronteira e gerar novo congelamento.

8 Originalidade e alcance dos direitos

A originalidade candidata está nas modificações da gramática e dos tipos e na implementação dos formatadores que organizam elementos de casos de uso em contrato JSON específico. O alcance do pedido deve se limitar às modificações e à seleção ou organização autoral, respeitando os direitos e avisos da base Apache-2.0.

A licença autoriza uso, modificação e redistribuição sob suas condições, mas sua suficiência documental para o processo brasileiro deve ser confirmada pelo NIT. O arquivo `LICENSE` deve acompanhar o pacote, e avisos de copyright existentes não devem ser removidos.

9 Classificação candidata

Categoria	Opções candidatas
Campo de aplicação	CO03 - linguagem artificial; IF02 - processamento de informação; IF10 - processamento de dados
Tipo de programa	DS01 - ferramenta de suporte ao desenvolvimento; DS05 - biblioteca de rotinas; DS06 - apoio à programação; TC01 - aplicações técnico-científicas
Linguagens	TypeScript, JavaScript e PEG.js
Forma de apresentação	código-fonte e documentação técnica identificados por SHA-256

As opções são propostas técnicas e devem ser confirmadas nas tabelas vigentes do INPI e pelo NIT.

10 Privacidade e configuração externa

A auditoria da branch não identificou CPF, telefone, endereço pessoal, caminho local, chave privada ou valor de serviço privado no código desta versão. Referências a `secrets` em workflows representam nomes de segredos, não seus valores.

- tokens NPM e GitHub devem permanecer nos mecanismos de segredo das plataformas;
- `.env`, configurações locais e credenciais não integram o pacote;
- URLs da obra originária e metadados de licença permanecem como proveniência;
- o ZIP final deve ser novamente verificado antes do cálculo do hash.

11 Pacote depositável e forma de entrega

11.1 Conteúdo a incluir

- arquivos do núcleo listados na Seção 4.3;
- arquivos necessários para compilar a gramática e o TypeScript;
- `package.json` e lockfile;

- testes e exemplos sintéticos pertinentes;
- `LICENSE` e demais avisos aplicáveis;
- README com instalação, entrada, saída e exemplo;
- esta especificação e manifesto de arquivos;
- relatório ou patch que documente `966a72e..f317b79`.

11.2 Conteúdo a excluir

- `.git` e branches não pertencentes à versão;
- `node_modules`, `dist`, `build`, cobertura e caches;
- tokens, `.env` e configurações locais;
- arquivos de classes, sequências ou atividades posteriores;
- dados e resultados empíricos;
- código do Produto 3 ou de outros produtos.

11.3 Artefatos de entrega

1. commit ou tag da versão congelada;
2. ZIP determinístico;
3. manifesto de arquivos e relatório de proveniência;
4. SHA-256 do ZIP;
5. licença e avisos;
6. esta especificação e instruções de reconstrução;
7. cópia exata do ZIP em ao menos dois locais controlados.

12 Verificação e limitações

Item	Estado	Ação antes do hash
Fronteira autoral	delimitada por `966a72e..f317b79`	gerar relatório final do diff
Dados pessoais e segredos	não encontrados no snapshot auditado	repetir varredura no ZIP
Dependências Node	declaradas nos manifestos	reinstalar de forma limpa
Compilação e testes	não comprovados no ambiente final	executar `npm ci`, build e testes
Exemplo de caso de uso	existe entrada de apoio	transformar em teste reproduzível
Licença e proveniência	Apache-2.0 e upstream identificados	validar documentos com o NIT

13 Resumo digital

Campo	Valor
Algoritmo	SHA-256
Nome sugerido	`produto4-plantuml-parser-hsl-usecase-v1.0.0.zip`
Resumo digital	**PENDENTE** - gerar após manifesto, testes e congelamento
Commit ou tag do pacote	**PENDENTE**
Custódia do arquivo	**PENDENTE** DE DEFINIÇÃO INSTITUCIONAL

14 Pendências institucionais

- confirmar autoria formal das modificações e titularidade correspondente;
- confirmar como a obra originária ou seus titulares devem constar no pedido;
- validar a Apache-2.0 como documento de autorização e os anexos necessários;
- decidir data de criação e publicação;
- confirmar classificações do INPI;
- aprovar a fronteira restrita a casos de uso;
- gerar o pacote final e seu SHA-256.